Chap 4 - PHP

L2 informatique - Programmation Web

Plan du Cours

1. Introduction
2. Variables, fonctions, structures de contrôle
3. Formulaires
4. Accès aux données des formulaires
5. Accès à une base de données mysql
6. Cookie, Session

Outline

1. Introduction
2. Variables, fonctions, structures de contrôle
3. Formulaires
4. Accès aux données des formulaires
5. Accès à une base de données mysql
6. Cookie, Session

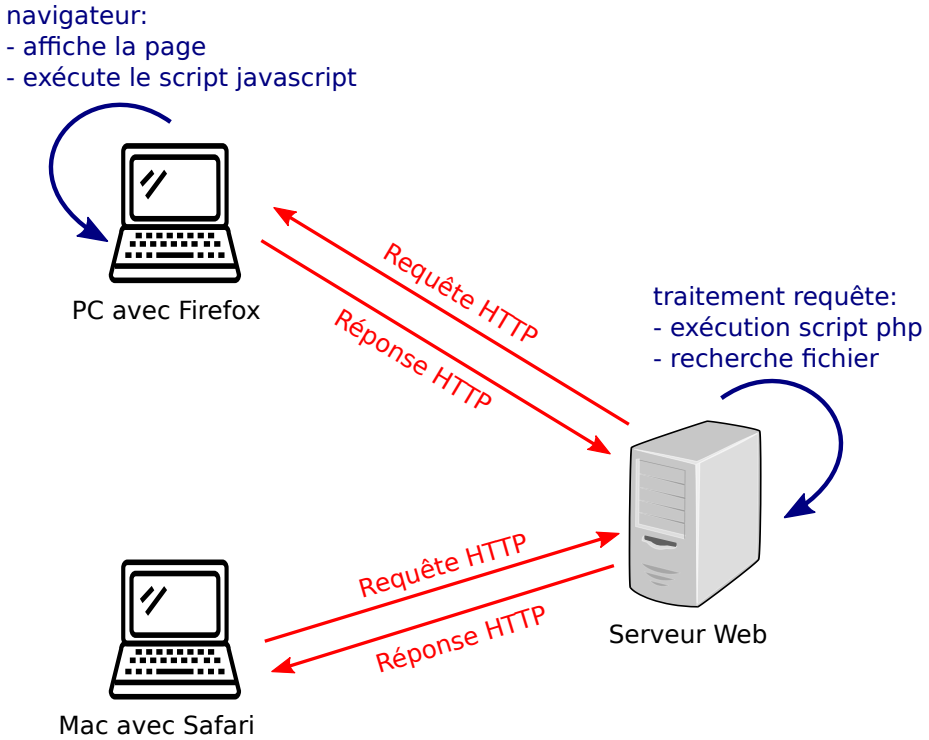
Historique

- 1994. Première version par Rasmus Lerdorf (à l'origine : une bibliothèque logicielle en C).
  - PHP/FI = Personal Home Page tool / Form Interpreter.
- 1997. Réimplémentation complète par deux étudiants (Andi Gutmans et Zeev Suraski)
  - PHP = Hypertext Preprocessor
- 2015. sortie de la version 7.
- 2020. sortie de la version 8.
- 2022. utilisé sur 74.7% des serveurs Web.

Source `https://w3techs.com`

PHP : caractéristiques

- Le client fait une demande d'un fichier `page-web.php`, le serveur exécute le script php et envoie le produit (sortie standard) du script.



- Le script php n'est pas visible/accessible par le client.
- Serveurs Web les plus populaires Nginx et Apache.

PHP : caractéristiques

- Fichier avec extension .php
  - Entièrement en PHP.
  - Mélange de HTML et de PHP.
- Générer des pages dynamiquement : intègre des données/éléments (fichiers ou base de données):
  - Des pages html
  - Des styles CSS
  - Des script JavaScript
- Gérer une session client.
- Sécuriser les données : elles sont sur le serveur, leur accès est donc contrôlé par le serveur.

PHP : caractéristiques

- Langage impératif
- Langage orienté objet
- Langage faiblement typé
- Langage sous licence libre

Syntaxe inspirée du C:

```
1  <?php
2   $films = array('Taxi Driver','Spring Breakers','Parasite','12
       Monkeys');
3   echo "\n" . '<ul>';
4   foreach ( $films as $film ) {
5     echo "\n\t" . '<li>' . $film . '</li>';
6  }
7  echo "\n" . '</ul>';
8  ?>
```

Code qui génère une liste non ordonnée.

Commentaires en PHP

Trois syntaxes pour les commentaires:

- Bloc entre /* et */.
- Fin de ligne après //.
- Fin de ligne après #.

```
1  <?php
2  $langage='PHP'; // Definition d'une variable
3  /* $langage = ' OCaml ';
4  echo 'Je deteste le!' . $langage . '!';
5  */
6  echo 'Le' . $langage . ",c'est bien !";
7  # Affiche "Le PHP, c'est bien !"
8  ?>
```

PHP est exécuté coté serveur il ne seront donc pas visible sur la page web générée.

Activation et désactivation des erreurs

En mode développement, il est recommandé d'activer les erreurs.

- Les messages d'erreurs sont afficher dans la page HTML

```
1        <?php
2        // activation du report de toutes les erreurs
3        error_reportaing(E_ALL);
4        ?>
```

- Désactivation : en mode production il vaut mieux que les erreurs ne s'affichent pas sur la page HTML.

```
1           <?php
2        // desactivation du report de toutes les erreurs
3        error_reporting(0);
4        ?>
```

Outline

1. Introduction
2. Variables, fonctions, structures de contrôle
3. Formulaires
4. Accès aux données des formulaires
5. Accès à une base de données mysql
6. Cookie, Session

Les variables

- Le nom commence par $ (pour ne pas être confondues avec les fonctions) cédures et fonctions.
- Le nom est formée de lettres, chiffres ou _.
- Sensible à la casse `$AAA \neq $aaa`
- Le type est dynamique:

```
1  $x=5; // int
2  $x=5.3; // float
3  $x="une chaine de caractere";
4  $x=array(1,"aaa","bb","c");
```

- Recommandation : éviter les changements de type pour une variable.

Les nombres

- Les entiers:

```
1  // Nombres entiers
2  $a = 2046; $b = 300;
3  echo 2046 - 300;
4  echo intdiv (2046 , 300) ; // affiche 6 ( divison entiere )
```

- Les flottants:

```
1  // Nombres flottants
2  $a = 2.046; $b = 2.04e6 ;
3  echo $a + $b ; echo $a / $b ;
```

- Il y a les fonctions mathématiques usuelles (cos, sqrt, rand, floor, ceil, round, intdiv, ...). Pour `rand(val_min,val_max)` on génère un entier entre `val_min` et `val_max`.

Portée des variables

- La portée d'une variable est le script dans lequel elle définie
  - Elle peut être déclarée dans une section `<?php ... ,>` et utilisée dans une autre.
- Les variables sont supprimées à la fin de l'exécution du script.

Exemple.

```
1  <?php
2  echo '$nom =' . $nom . '<br>';
3  $nom='michael';
4  echo '$nom =' . $nom . '<br>';
5  ?>
```

Premier appel au script

```
nom=
nom=michael
```

Second appel au script

```
nom=
nom=michael
```

Fonction isset et unset

- La fonction isset(variable) indique (booléen) si la variable est définie ou pas.
- La fonction unset(variable) supprime la variable

```
1  <?php
2
3  $variable=3;
4  $def=isset($variable);
5  echo '$variable='.$variable.'<br>';
6
7  if( $def )
8  {
9    echo '$variable est definie <br>';
10 }
11 else
12 {
13   echo '$variable n\'est pas definie <br>';
14 }
15
16 unset($variable);
17 $def=isset($variable);
18 echo '$variable='.$variable.'<br>';
19 if( $def )
20 {
21   echo '$variable est definie <br>';
22 }
23 else
24 {
25   echo '$variable n\'est pas definie <br>';
26 }
27 ?>
```

Les chaînes de caractères : guillemets

Deux syntaxes:

- Guillemets simples : aucune variable n'est interprétée:

```
1        echo 'J aime le film \"Mon petit truc en plus\"';
2        // affichera  J aime "Mon petit truc en plus"
```

- Guillemets doubles : les variables et `'\n','\t'` interprétés:

```
1        $film="Mon petit truc en plus";
2        echo "J aime le film \"$film\" \n ";
3        // affichera  J aime "Mon petit truc en plus"
4        // suivi d'un retour a la ligne
5        echo "J\'aime le film \"${film}\" \n ";
```

Par défaut utiliser les guillemets simples.

Les chaînes de caractères : opérations

- On concatène avec ".."
- On affiche avec echo
- Pour une chaine $x, $x[i] est le i-eme caractères.
- Exemple : affichage de la date

```
1 <?php
2 $aujourdhui = date('d-m-Y h:i:s');
3 echo "\n" . '<p> Nous sommes le' . $aujourdui . '</p>';
```

- Autres fonctions usuelles :
  - strlen(str)
  - printf comme printf de C
  - strcmp(str1,str2)
  - substr(str,start,end) extrait une sous chaine
  - explode
  - strpos(str1,str2 renvoie la position où str2 apparaît dans str1.

  Pour une liste plus complète voir https://www.w3schools.com/php/php_ref_string.asp.

Les tableaux

- Tableau indicé :
  - déclaration : avec `array(element0,element1,...)` .
  - la taille n'est pas prédéfinie.
  - on accède aux éléments avec l'indice `$nom_tableau[indice]`.
- Tableau associatif
  - déclaration : avec `array(cle1=>valeur1,cle2=valeur2,...)` .
  - on accède avec `$nom_tableau[cle]`.

Exemple tableau

```php
<?php
// Ensemble de valeurs
$realisatrices = array( 'Ducournau', 'Despentes', 'Sciamma');
echo $realisatrices[1] . '<br>'; // Affiche 'Despentes '

// Cles => valeurs
$films = array(
    'Petite Maman' => array(
                        'real'=> 'Sciamma',
                        'sortie' => 2021 ,
                        'interpretation' => array( 'Sanz', '
                            Meurisse')
            ),
    'Jackie Brown' => array(
                         'real' => 'Tarantino',
                         'sortie' => 1997,
                         'interpretation' => array( 'Grier' , '
                             Jackson' , 'De Niro')
                          ),
    'Lalaland' => 'aucune info'
                 ) ;

echo $films['Jackie Brown']['interpretation'][2] . '<br>'; //
    Affiche 'De Niro'

?>
```

Les opérateurs arithmétiques sont similaires aux opérateurs en C.

Signe	Signification	Exemple
=	a la valeur affectation	x=5
+	addition	x + 3
-	soustraction	x - 3
*	multiplication	x*2
/	division	x /2
%	modulo reste de la division	x%5

Il y a aussi

- les opérateurs associatifs `+=, -=, *=, /=`
- Et les opérateurs d'incrémentation `++,--`

Opérateurs de comparaison, opérateurs logiques

Il y a huit opérateurs de comparaison (pour l'exemple x=11) (similaires au C)

Symbole	Nom	Exemple	Résultat
==	égal (avec transtypage)	x==11	true
<	inférieur	x<11	false
<=	inférieur ou égal	x<=11	true
>	supérieur	x>11	false
>=	supérieur ou égal	x>=11	true
!=	différent (avec transtypage)	x! =11	false

Voici les trois opérateurs logiques:

Symbole	Nom	Exemple	Résultat
&&	et	(cond1) && (cond2)	cond1 et cond2
||	ou	(cond1) || (cond2)	cond1 ou cond2
!	not	!cond	not cond

Structures de contrôles

Les structures de contrôles sont les mêmes que dans le langage C.

- Structure if..else

```
if (condition) { instructions;} else { instructions;}
```

- Structure for

```
for (i = 1;i <= 10;i++) { instructions; }
```

- Structure while

```
while(condition) {instructions;}
```

- Aussi switch

```
switch(condition) { case ..: ... default: ... }
```

Boucle foreach

Cette boucle parcours les élements du tableau `tab[i]` et affecte la valeur dans la variable de boucle donnée par le as

```
1        $films = array( 'Taxi Driver' , 'Spring Breakers' , 'Parasite'
               , ' 12 Monkeys') ;
2        foreach ( $films as $film )
3        {
4           echo $film . "<br>" ;
5        }
```

Fonctions

- Les fonctions sont similaires au C.
- Seule différence : il n'y a pas de typage!

```
1     <?php
2  function nom_fonction($arg1,...,$argn)
3  {
4     //...  instructions ...
5     return(valeur_renvoyee);
6  }
7  ?>
```

- Exemple : fonction d'élévation au carré

```
1   <?php
2  function carre($x)
3  {
4     return($x*$x);
5  }
6  ?>
```

Mode strict

- Dans PHP7 on peut utiliser un typage plus strict:

```
1  <? php
2  // Premiere instruction du script php:
3  declare ( strict_types =1) ;
4  ?>
```

- Conséquence : typage optionnel des fonctions (mais doit alors être valide !)

```
1  <? php
2  function carre(int $x) : int{
3  {
4     return($x*$x);
5  }
6  function affiche_film(string $titre, string $real, int $annee) :
       void{
7  echo "\n<p>$titre a ete realise par $real en $annee. <p>";
8    }
9    ?>
```

Valeurs des arguments par défaut

Si un argument n'est pas spécifié, on utilise la valeur par défaut

```
1          <? php
2         function affiche_film(string $titre, string $real, int $annee
              =2025) : void{
3           echo "\n<p>$titre a ete realise par $real en $annee. <p>";
4         }
5         affiche_film('Taxi Driver', 'Scorcese' 1976);
6         affiche_film('Un petit truc en plus', 'Artus',2024);
```

Affichera

```
<p> Taxi Driver a ete realise par Scorcese en 1976  </p>
<p> Un petit truc en plus a ete realise par Artus en 2024  </p>
```

Passage par référence

On peut passer un argument par référence:

- la fonction travaillera alors sur la donnée d'origine
- et pas sur une copie.

```
1   function increment($n)
2   {
3     $n++;
4   }
5   function increment_ref(&$n)
6   {
7     $n++;
8   }
9   $a=21;
10  increment($a);
11  echo $a . '<br>'; // affiche 21
12  increment_ref($a);
13  echo $a . '<br>'; // affiche 22
```

Un tableau est toujours passé par copie (sauf si on spécifie le passage par référence).

Portée des variables et variables globales

- Les variables déclarées dans une fonction ne sont pas accessibles en dehors de la fonction.
- On peut déclarer une variable globale avec le mot clef global
- Si une variable locale et une variable globale ont le même nom, la variable locale est prioritaire.
- On peut accéder à des variable globale avec $GLOBALS['nom_variable'].

```
1   function f()
2   {
3       $a=2;
4       echo "dans f, \$a=" . $a . "<br>";  // affiche 2
5       echo "dans f \$GLOBALS['a']=" . $GLOBALS['a'] . "<br>";  //
            affiche 3
6       global $b;
7       $b=5;
8   }
9   $a=3;
10  echo "en dehors de f, \$b=$b" . "<br>"; // affiche rien
11  f();
12  echo "en dehors de f, \$b=". $b . "<br>"; // affiche 5
13  ?>
```

Variables super globales

L'interpréteur maintient les tableaux associatifs suivants:

`$GLOBALS`	contient toutes les variables globales
`$_COOKIE`	tableau associatif pour gérer les sessions
`$_GET`	données transmises par une requête GET
`$_POST`	données transmises par une requête POST
`$_FILES`	données sur les fichiers ouverts
`$_ENV`	données sur les variables d'environnement

Report d'erreurs, deboguage

Gestion d'erreur en développement :

- En phase de développement, il est recommandé d'activer l'affichage d'erreurs.
  - Activer/Désactiver le report d'erreur:

```
1        // Desactive le report d'erreur
2        error_reporting(0);
3        // Report de toutes les erreurs
4        error_reporting(E_ALL);  // parametre donne le niveau
             des erreurs reportees
```

    La configuration du serveur peut ignorer ces instructions.
  - Gestion d'une erreur :

```
1    $test=2;
2    if ($test>=1) {
3       die("La valeur doit etre inferieure a 1");
4    }
```

- En phase de production il ne faut pas afficher les erreurs (fuites d'information).

Erreur d'interprétation :

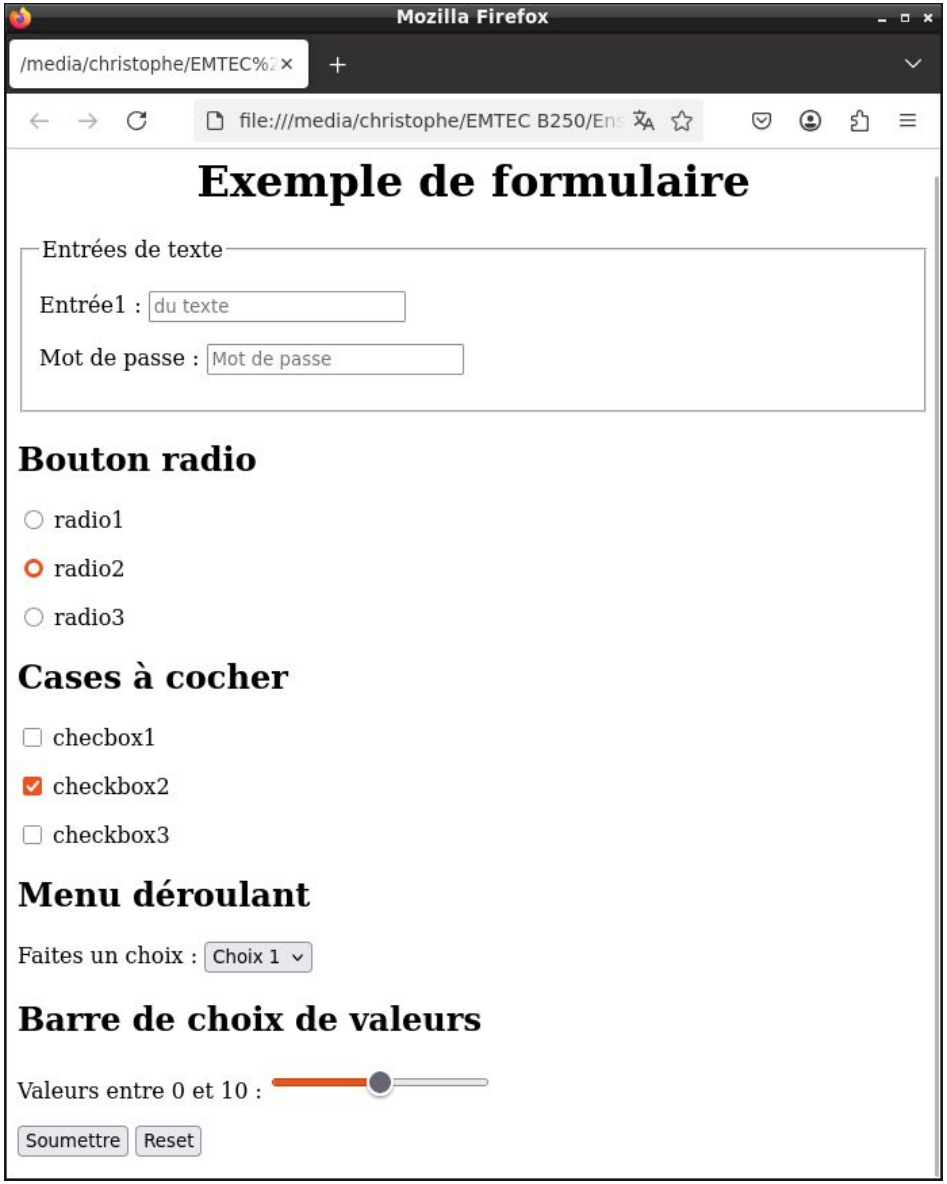
- Les erreurs d'interprétations font planter entièrement l'exécution d'un script, le serveur web envoie alors un fichier vide.
- Pour avoir les erreurs d'interprétation: exécuter le script dans le terminal : `php script.php`.
- Pour les variables GET ou POST il faut les fixer dans le script lui-même.

Outline

1. Introduction
2. Variables, fonctions, structures de contrôle
3. Formulaires
4. Accès aux données des formulaires
5. Accès à une base de données mysql
6. Cookie, Session

Les formulaires

- Les formulaires permettent de récupérer des informations auprès des visiteurs du site web.
- Le visiteur d'une page peut à travers un formulaire
  - Rentrer du texte.
  - Cocher un élément dans des choix multiples.
  - Choisir un élément dans une liste.


Mozilla Firefox
/media/christophe/EMTEC%2
file:///media/christophe/EMTEC B250/En
Exemple de formulaire
Entrées de texte
Entrée1 :
du texte
Mot de passe :
Mot de passe
Bouton radio
radio1
radio2
radio3
Cases à cocher
checbox1
checkbox2
checkbox3
Menu déroulant
Faites un choix :
Choix 1
Barre de choix de valeurs
Valeurs entre 0 et 10 :
Soumettre
Reset

Formulaire en HTML - Deux principales balises

- La balise <form> indique le début et la fin d'une zone de formulaire. Indique aussi comment il sera envoyé les données au serveur (GET ou POST).
- La balise <input> (qui se trouve à l'intérieur de <form>) permet d'insérer les différents éléments d'un formulaire.
- Le choix du type d'<input> se fait grâce à l'attribut type.

Type	Element
text	petite zone d'entrée de texte
textarea	grande zone d'entrée de texte
checkbox	case à cocher
radio	bouton radio
hidden	élément caché
submit/reset/button	différent type de bouton

Pour texte il y a des variantes (color,date,email,password,tel,...)

Balises complémentaires, entrées texte

Balises complémentaires

- <label> avec l'attribut for on associe l'élément du formulaire qui correspond (for="id de l'input référencé").
- <fieldset> permet d'avoir un meilleur rendu.
- <legend> définit le sous-titre de l'élément <fieldset>.

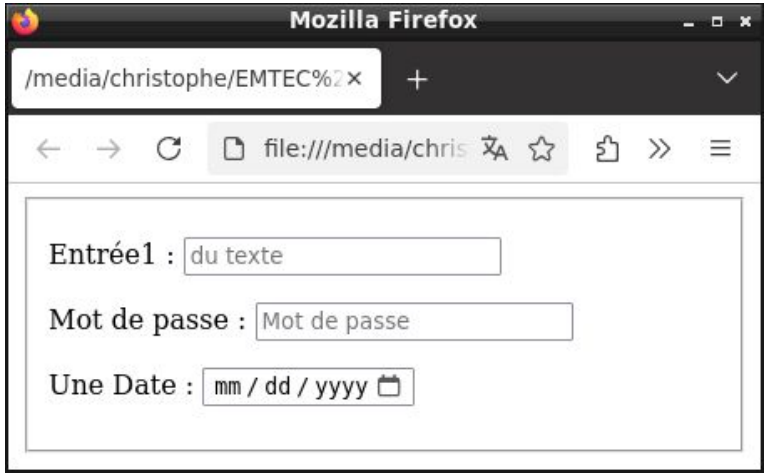
Entrée texte se déclare avec :

```
1 <input type="text" name="nom" default="Martin">
```

- name = est le nom de la variable dans laquelle sera stockée la valeur rentrée;
- placeholder = valeur affichée par défaut ;
- Les propriétés CSS size ajuste la taille de la zone, maxlength =nombre de caractères maximal autorisé ;

Exemple d'entrées texte

```html
<fieldset>
  <p>
    <label for="texte1"> Entree1 : </label>
    <input type="text" name="texte1" id="texte1" placeholder="du texte">
  </p>
  <p>
    <label for="mdp"> Mot de passe :</label>
    <input type="passwd" name="mdp" id="mdp" placeholder="Mot de passe">
  </p>
  <p>
    <label for="date"> Une Date :</label>
    <input type="date" name="date" id="date">
  </p>
</fieldset>
```



Liste déroulante

Se fait avec les balises <select> et <option>

```
1    <select name="label" size="n">
2        <option selected> choix1 </option>
3        <option> choix2 </option>
4        ...
5    </select>
```

Attributs de l'élément <select> :

- name = nom pour la donnée ;
- size = nombre de lignes affichées pour une liste multiple ; la liste ;

Attributs de l'élément <option> :

- value définit la valeur passée au script ;
- selected pré-sélection d'un item de la liste ou valeur par défaut du menu.

Menu déroulant - Exemple

Le code suivant

```
1 <label for="choix"> Faites un choix :</label>
2 <select name="listeexemple" id="choix" size="1">
3 <option selected="true"> Choix 1 </option>
4 <option> Choix 2  </option>
5 <option> Choix 3  </option>
6 </select>
```

Donne

Cases à cocher : type="checkbox"

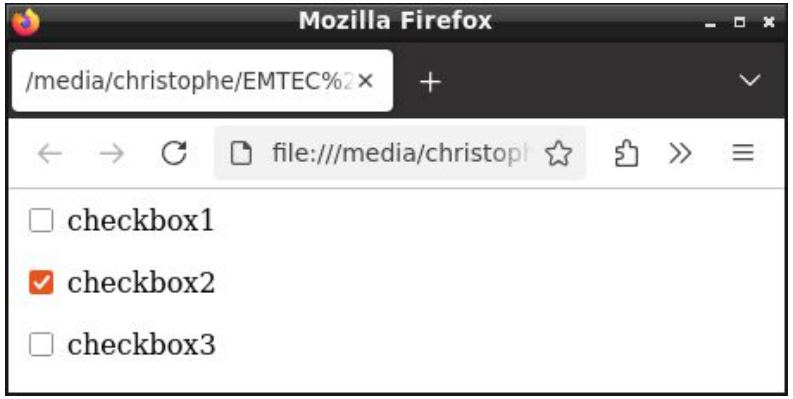
```
<input type="checkbox" name="label" value="texte">
```

Attributs :

- name pour nommer les éléments de la case à cocher (quand le formulaire est soumis seules les cases cochées sont envoyées au serveur) ;
- value pour spécifier la valeur à envoyer, par défaut la valeur "on" est envoyée ;
- checked (="true"/"false") positionne par défaut le bouton en mode validé.

Exemple :

```
1 <p> <input type="checkbox" name="checkbox1"> <label> checkbox1 </
      label> </p>
2 <p> <input type="checkbox" name="checkbox2" checked="true"> <label>
      checkbox2  </label> </p>
3 <p> <input type="checkbox" name="Checkbox3"> <label>checkbox3  </
      label></p>
4 </p>
```



Boutons radio : type="radio"

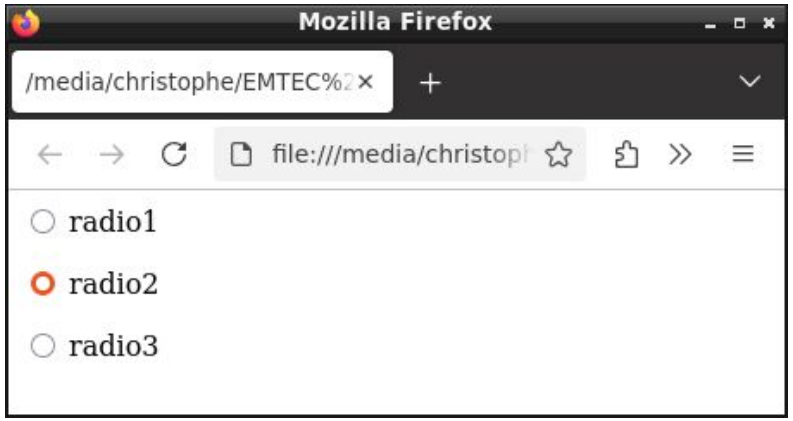
```
<INPUT type="radio" name="label_groupe" checked value="label">
```

Attributs :

- name pour nommer un groupe de boutons (il faut donner le même nom à tous les boutons radio du même groupe pour qu'ils soient exclusifs) ;
- value pour spécifier la valeur à renvoyer ;
- checked (="true"/"false") pour sélectionner par défaut.

Exemple :

```
1 <p> <input type="radio" name="exemple"> <label> radio1 </label> </p>
2 <p> <input type="radio" name="exemple" checked="true"/> <label>
      radio2 </label> </p>
3 <p> <input type="radio" name="exemple"> <label> radio3 </label> </p>
```



Barre de choix de valeurs

Avec

```
<input type="range">
```

Exemple :

```
1 <label for="valeurs"> Valeurs entre 0 et 10 :</label>
2 <input type="range" id="valeurs" name="valeurs" min="0" max="10">
```

Différents Boutons

Les boutons submit et reset ont un effet sur tous les champs d'un <form>

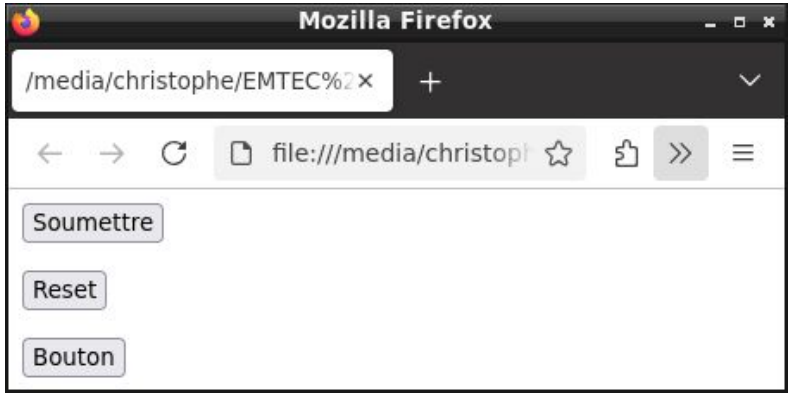
- Soumission du formulaire : <input> avec type="submit", la soumission est faite sur le script action="script.php" de <form> .
- Effacer le formulaire : <input> type="reset", réinitialise les valeurs du formulaire.
- Autre bouton (pas forcément dans un formulaire) : <input> avec type="button" (gèrer le click avec JavaScript).

Attributs :

- name = sert nommer le bouton à l'intérieur du formulaire ;
- value = texte inscrit sur le bouton.

Exemple :

```
1 <p>  <input type="submit" value="Soumettre"> </p>
2 <p>  <input type="reset" value="Reset"> </p>
3 <p>  <input type="button" value="Bouton"> </p>
```



Soumission du formulaire

- Dans la balise <form> on choisit quel type d'envoi est fait (get/post) avec l'attribut method:

  `method="get"`

  ou

  `method="post"`
- Dans la balise <form> on indique où les données seront envoyées avec

  `action="script-traitement-formulaire.php"`
- Lorsque l'utilisateur appuie sur submit une requête (GET ou POST) est faite à un script (php ou autre sur le serveur).
- Lors de cette requête le client envoie les données identifiées par le name de chaque input du <form>.
- Le client attend alors la réponse du script.

Outline

1. Introduction
2. Variables, fonctions, structures de contrôle
3. Formulaires
4. Accès aux données des formulaires
5. Accès à une base de données mysql
6. Cookie, Session

Avec une requête GET les données du formulaire sont passées dans l'url:

`http://www.unsiteweb.php?cle1=valeur1&cle2=valeur2&...`

On transmet les valeurs de différentes variables:

- Variables de noms `cle1` qui a la valeur `valeur1`.
- Variables de noms `cle2` qui a la valeur `valeur2`.
- ...

Exemple.

`https://www.bad-movies.com/sp/search?film=batman&year=2000`

qui deux variables `film` qui vaut "batman" et `year` qui vaut 2000.

Accès valeurs données pas par le GET

L'url suivante

`http://www.unsiteweb.php?cle1=valeur1&cle2=valeur2&...`

est transformé en tableau associatif `$_GET`.

- `$_GET['cle1']` contient `valeur1`
- `$_GET['cle2']` contient `valeur2`
- ...

Exemple :

- La page avec un formulaire avec 2 entrées:

```
1  <html> <body>
2  <form name="exploit-sql" method="POST" action="http://localhost/
       exemple-get.php">
3    Film : <input  name="movie"> <br>
4    Realistateur : <input  name="producer"><br>
5  <input type="submit">
6  </form>
7  </body> </html>
```

- La page qui traite les données du formulaire :

```
1  <?php
2  echo "<html> <body>";
3  echo "Voici les donnees recuperees :"
4  echo "<ul>";
5  echo "<li> Titre :" . $_GET['movie'] . "</li>";
6  echo "<li> Realisateur :" . $_GET['produceur'] . "</li>";
7  echo "</ul>";
8  echo "</body></html>";
9  ?>
```

Encodage d'une url en PHP

- Les caractères spéciaux "/", "&", " " doivent être recodés.
- On utilise la fonction `urlencode`.

  Exemple : une balise <a> dans la sortie PHP:

```
1  echo '<a href="' . $_SERVER['PHP_SELF']. '?search?film=' .
       urlencode($film)
2  . '&producer=' . urlencode($realisateur) . '"> Lancer la
       recherche </a>";
```

  où `$_SERVER['PHP_SELF']` est le serveur web qui traite la requête.

- Le résultat est

`https://www.monsite.fr/index.php?cours=Un%20petit%20truc&realisateur=artus`

Formulaire et traitement sur la même page php

- On peut traiter un formulaire par lui même: on utilise `isset` (teste si une variable a été définie).

```
1   <?php
2  if( isset( $_GET['texte'] )
3  {
4     echo "Vous avez rentre . $_GET['texte'] . "<BR>";
5  }
6  else
7  {
8     echo "<html> <body>";
9     echo "<form name=\"exploit-sql\" method=\"GET\" action=\"http
          ://localhost/exemple-isset.php\">";
10    echo "Entrez du texte : <input  name=\"movie\"> <br>"
11    echo "<input type=\"submit\">;
12    echo "</form>";
13    echo "</body> </html>";
14 }
15 ?>
```

- On peut préremplir par des données déjà envoyées avec les sessions.
- Lorsque l'on recharge une page de formulaire les données sont perdues.

Méthode POST

Les données peuvent être transmise par la méthode POST

- Les clés/valeurs n'apparaissent plus dans l'url.
- Les clés/valeurs se trouvent dans le corps du message HTTP.

Seules différences avec GET :

- méthode POST dans le formulaire
- Dans le script de traitement, accès aux données via le tableau `$_POST`.

Méthode POST : exemple

- La page avec le formulaire avec 2 entrées:

```
1  <html> <body>
2  <form name="exploit-sql" method="POST" action="http://localhost/
       exemple-get.php">
3    Film : <input  name="movie"> <br>
4    Realistateur : <input  name="producer"><br>
5  <input type="submit">
6  </form>
7  </body> </html>
```

- La page qui traite les données du formulaire :

```
1  <?php
2  echo "<html> <body>";
3  echo "Voici les donnees recuperees :"
4  echo "<ul>";
5  echo "<li> Titre :" . $_POST['movie'] . "</li>";
6  echo "<li> Realisateur :" . $_POST['produceur'] . "</li>";
7  echo "</ul>";
8  echo "</body></html>";
9  ?>
```

Outline

1. Introduction
2. Variables, fonctions, structures de contrôle
3. Formulaires
4. Accès aux données des formulaires
5. Accès à une base de données mysql
6. Cookie, Session

Motivation

- PHP permet de générer des pages html dynamiquement, qui s'adapte aux données utilisateurs de la session.
- PHP ne permet de stocker des données sur du long terme (sauf dans des fichiers).

Solution : bases de données SQL

- Base de recherche des moteurs de recherche.
- Posts et commentaires utilisateurs.
- Identifiants et mots de passes.
- Historique de navigation.
- etc.

La structuration des données SQL facilite la recherche d'information.

Suite logicielle utilisée en TP

On utilisera la suite LAMP (Linux Apache, MySQL, PHP):

- interpréteur PHP
- mysql pour la gestion de base de données SQL
  - Système de gestion de base de données relationnelles SQL.
  - Distribué sous licence libre (fork opensource est mariadb).
  - Très efficace en lecture (mais moins en écriture).
  - Alternative: Oracle, PostgeSQL, sqlite.
- Apache pour le serveur web

Se connecter au serveur MySQL

- PHP a une interface pour faire des requêtes sur un serveur MySQL.
- Voici comment se connecter à une base de données MySQL :

```
1  $serveur = 'localhost';
2  $utilisateur = 'root';
3  $mot_passe = 'password';
4  $base = 'nom_de_la_base';
5  $connexion = mysqli_connect($serveur, $utilisateur, $mot_passe,
       $base);
6  if (!$connexion) {
7    // gestion de l'erreur
8  }
9  // code php pour les requetes sur la BD MySQL
10 // ...
11 mysqli_close($connexion);
```

Principe : Ne pas afficher les erreurs par défaut mais les gérer depuis PHP :

Requête création, modification du schéma ou des données

La procédure `mysqli_query` soumet la requête et renvoie `false` si elle a échoué, `true` sinon.

```
1  $ma_requete_sql='create table ...'; // par exemple ou bien insert
       ... ou bien update ... etc
2  $resultat=mysqli_query($connexion, $ma_requete_sql);
3  if (!$resultat) {
4      // si on veut eventuellement traiter l'erreur
5  }
```

Requête SELECT

- Même syntaxe : mais ici la requête est un SELECT.
  `$resultat=mysqli_query($connexion,$requete)`
- `$resultat` contient les différentes lignes de la table resultat du select.

```
1  $ma_requete_sql='" SELECT nom , annee FROM Films LIMIT 5"';
2  $resultat=mysqli_query($connexion, $ma_requete_sql);
3  if (!$resultat) {
4      // si on veut traiter les erreurs
5  }
6  echo  mysqli_num_rows($resultat) . "lignes dans le resultat <br>";
7  while ($ligne=mysqli_fetch_array($resultat)) {
8      echo $ligne [ 'nom'] . ' a ete realise en ' . $ligne [ 'annee'
           ];
9  }
10 my sqli_free _result($resultat); // on libere le  resultat
```

Variante pour la requête SELECT

- Pour le parcours des lignes du résultat, on peut obtenir un tableau indicé.
- On l'obtient en passant `MYSQLI_NUM` en second argument à `mysqli_fetch_array`

```
1  while ($ligne=mysqli_fetch_array($resultat, MYSQLI_NUM)) {
2      // $ligne[0] est le premier attribut (colonne) de la ligne
3      // $ligne[1] est le second attribut (colonne) de la ligne
4  }
```

- Si on passe `MYSQLI_ASSOC` on aura le tableau associatif (par défaut).

Codages de caractÃ¨res

Notez que vous avez trois langages traités par des programmes différents:

- Codage en PHP
- Codage en MySQL
- Codage en HTML

Il vaut mieux avoir tout en UTF-8 pour éviter les incohérences entre les 3.

- Pour MySQL : définir la base en `'utf8_unicode_ci'`.
- En PHP, préciser le codage lors de la connexion:

```
1  $connexion->set_charset("utf8");
```

- En PHP, pour envoyer un fichier (par exemple HTML) au bon format:

```
1  header ('Content-type: text/html ; charset=utf-8');
```

Outline

1. Introduction
2. Variables, fonctions, structures de contrôle
3. Formulaires
4. Accès aux données des formulaires
5. Accès à une base de données mysql
6. Cookie, Session

Les cookies

Problème :

- Le protocole HTTP est un protocole sans état : les requêtes à un serveur web sont faites les unes indépendemment des autres.
- Comment faire pour maintenir des informations sur la navigation d'un internaute ?
- Solution : cookies et session.

Principe des cookies :

- Un cookie est un petit fichier déposé sur le poste de l'internaute.
- Les cookies sont automatiquement renvoyés au serveur web pour chaque nouvelle requête.
- Ils servent à maintenir des informations récentes de la navigation.
- Ils peuvent persister à une session web.

Création et accès à un cookie

- Création. On utilise la fonction `setcookie`

  `setcookie(chaine nom, chainevaleur)`

  Il y a des champs optionnels :

  `setcookie(chaine nom, chaine valeur, entier experiation, chaine chemin, chaine domaine, bool securise, bool http, option)`

- Exemple de création.

```
1  if(!isset($_COOKIE['id_internaute'])
2  {
3     setcookie ( 'id_internaute' , ' 12345 ') ;
4  }
```

Accès à un cookie :

```
1  if(!isset($_COOKIE['id_internaute'])
2  {
3     echo "Identifiant de session:" + $_COOKIE['id_internaute'];
4  }
```

Durée de vie d'un cookie

- Par défaut le cookie ne survie pas à la session.
- On peut augmenter la durée de vie avec le champs expiration.

Exemple : page1.php avec un lien vers page2.php

```
1 <?php
2 $ok1 = setcookie('prenom','Christophe');
3 $ok1 = setcookie('nom','Negre',time()+30*24*3600); // 30 jours
4 ?php>
5 <html> <body>
6 <a href="page2.php" lien vers page 2 </a>
7 </body> </html>
```

page2.php

```
1 <?php
2   echo "cookie prenom : " + $_COOKIE['prenom'];
3   echo "cookie nom : " + $_COOKIE['prenom'];
4 ?>
```

page2.php affiche

- Session en cours:

  `cookie prenom : Christophe`
  `cookie nom : Negre`
- Session 1 heure après:

  `cookie prenom :`
  `cookie nom : Negre`

Que mettre dans un cookie ?

- identifiant de session : le plus fréquent.
- préférence de session
- contenu d'un panier de site de commerce ligne.

Les données de session (préférence, panier) peuvent être stockées dans une base de données.

Gestion des sessions

Dans un certain nombre de cas (achat en ligne, authentification), une session est obligatoire :

- Pour pouvoir facturer la liste des produits à la fin.
- Pour limiter l'accès à des données confidentielles.

La solution est d'utiliser les sessions php

- Par défaut: elle s'appuie sur des cookies pour identifier la session
- Peut être refusé par l'internaute: les données de session sont alors transmises dans l'url.

Session : principe

php propose un ensemble de fonction pour faciliter la gestion des sessions:

- Un identifiant unique attribué au début de la session.
- L'identifiant est transmis de page en page (par cookie ou par url)
- Les données de session sont chargées au début du script et mis à jour à la fin d'un script php.

Pécautions : les données confidentielles (mot de passe, numéro de carte bancaire, etc) ne doivent pas être conservée dans les données de session ou de cookie.

fonction php pour la gestion de session

- `session_start` : ouverture/réactivation de session.
- `session_id` : renvoie l'id d'une session.
- `session_name` : donne le nom de la variable pour la session id.
- `session_abort` : annule les modifications des données de session et la termine.
- `session_reset` : réinitialise les données de session.
- `session_destroy` : supprime la session.
- `session_status` : donne le status actuel de la session.

Démarrage d'une session

```
1   <?php
2     session_start();
3     if( !isset($_SESSION['nb_visites']))
4     {
5      $_SESSION ['nb_visites']=0;
6      else
7      {
8      $_SESSION ['nb_visites']++;
9      }
10    }
11    echo $_SESSION['nb_visites'];
12  ?>
```

Identifiant et nom de variable de session

- Dans le script suivant on voit que l'identifiant de session est fixé par l'appel à `session_start`.
- `session_name` lui est fixé par la configuration du serveur, et ne change jamais.

```
1  <?php
2  $id_avant=session_id();  // id avant de definir l'identifiant
3  $name_avant=session_name();  // name avant de definir l'identifiant
4  session_start();
5  $id_apres=session_id(); // id apres le debut de la session
6  $name_apres=session_name(); // id apres le debut de la session
7  $actuel=date("H:i:s");// date/heure actuelle
8  echo "Heure :$actuel <br>";
9  echo "Avant, id : $id_avant, name : $name_apres <br>";
10 echo "Apres, id : $id_apres, name : $name_apres <br>";
11 ?>
```

Voici les résultats obtenus à deux moments différents:

- Première exécution:

```
Heure :12:42:01
Avant, id : , name : PHPSESSID
Après, id : lojd7qc6u81b7esgjl4np4ne2s, name : PHPSESSID
```

- Rechargement une minute après :

```
Heure :12:43:22
Avant, id : , name : PHPSESSID
Après, id : lojd7qc6u81b7esgjl4np4ne2s, name : PHPSESSID
```

La fonction session_abort annule les modifications de variables de session.

- Page 1:

```
1  <h1> session-abort-page1.php </h1>
2  <?php
3  session_start();
4  $_SESSION['nom']="Dupont";
5  echo "session id :" . session_id() . "</br>";
6  echo "nom :" . $_SESSION['nom'] . "</br>";
7  ?>
8  <a href="session-abort-page2.php"> lien vers page 2 </a>
```

- Résultat :

```
session-abort-page1.php
session id :lojd7qc6u81b7esgjl4np4ne2s
nom :Dupont
lien vers page 2
```

Session abort : suite

- Page 2:

```
1  <h1> session-abort-page2.php </h1>
2  <?php
3  session_start();
4  echo "avant, nom : " . $_SESSION['nom'] . '<br>';
5  $_SESSION['nom']="?";
6  session_abort();
7  echo "session id :" . session_id() . "</br>";
8  echo "apres, nom : " . $_SESSION['nom'] . '<br>';
9  ?>
10 <a href="session-abort-page3.php"> lien vers page 3 </a>
```

- Résultat :

```
avant, nom : Dupont
session id :lojd7qc6u81b7esgjl4np4ne2s
apres, nom : ?
lien vers page 3
```

- Page 3:

```
1  <h1> session-abort-page3.php </h1>
2  <?php
3  session_start();
4  echo "session id :" . session_id() . "</br>";
5  echo "nom : " . $_SESSION['nom'] . '<br>';
6  ?>
```

- Résultat :

```
session id :lojd7qc6u81b7esgjl4np4ne2s
session nom : Dupont
```

L'exemple montre que ce n'est effectif que dans le page suivant session_abord.

Session reset, destroy

- `session_reset` fait la même chose que `abort` mais c'est effectif immédiatement.
- `session_destroy` Cette fonction supprime toutes les variables de session (mais pas effectif immédiatement) et supprime la session. Mais ne supprime pas les cookies (donc l'identifiant reste dans les cookies).

Pour supprimer complètement une session: après un destroy réinitialiser vous même le tableau de cookies et le tableau de session.

Synthèse et références

Les choses importantes :

- Savoir faire un formulaire en html et le traiter avec un script php.
- Savoir faire des requêtes sur une base de données mysql et les incorporer dans une page html.
- Savoir gérer une session php.

Références :

- Le cours Programmation Web de Etienne André - Université Paris Nord.
- PHP8, Développez un site web dynamique et interactif, Olivier Heurtel, Ed. Ressources informatiques.
- Le site `https://www.w3schools.com`.